Contents

1. Tutorial Description

1.1 Python is a general-purpose programming language running on powerful hardware like desktops and servers. MicroPython is a streamlined version of Python specifically designed for microcontrollers and embedded devices.

1.2 This tutorial uses MicroPython for programming. The micro:bit does not have a dedicated NEC infrared MicroPython library. If using Pin to read NEC infrared signals, the reading speed is too slow, causing severe signal loss. Therefore, this tutorial cannot implement NEC infrared remote control.

1.3 micro:bit V1 has only 1 core, so it cannot run Bluetooth while running MicroPython. The latest micro:bit V2 has 2 cores and theoretically can run MicroPython and Bluetooth simultaneously, but this feature is still under official testing. For specific information, please check:

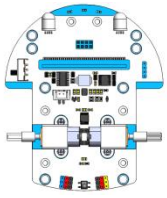
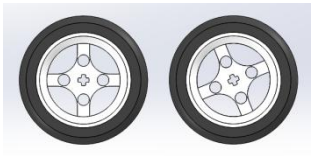
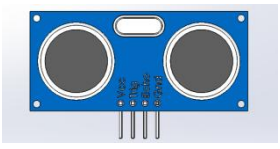
https://microbit-micropython.readthedocs.io/en/v2-docs/ble.html

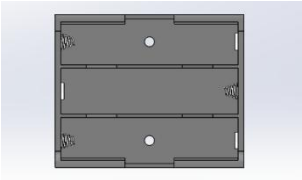
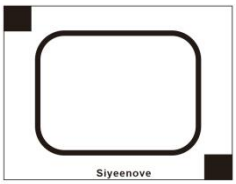
For the above reasons, this tutorial cannot implement Bluetooth control functionality. In the future, we will update our tutorials accordingly as official debugging information becomes available!

2. Pybit Introduction

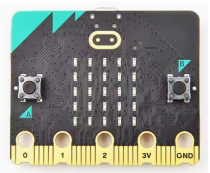
The Pybit, developed by Siyeenove, is a micro:bit car designed around maker education and DIY, with very convenient installation. It integrates an ultrasonic module, RGB lights, infrared receiver sensor, and infrared line-following sensor. The car body has Lego mounting holes and IO expansion ports, making it very convenient to connect various types of modules. It is suitable for use by everyone from children aged 7+ to university students and electronics enthusiasts.

2.1 Components List

PCB Chassis	Wheel	Ultrasonic Sensor
		

AA Battery Holder	IR Remote	Map
		
3M Adhesive Tape		
		

Note: AA batteries and the micro:bit v2.x.x mainboard need to be purchased separately.

AA Batteries	micro:bit - v2.x.x
	

2.2 Specifications

Mainboard Used: micro:bit - v2.x.x

Programming Methods: MakeCode and Python Editor

Battery Connector Type: XH2.54 2P, Input Voltage 3--5.5V, default uses 3 AA batteries in series.

Infrared Remote Type: NEC, Remote control distance ≧5 meters.

Motor Type: GA12-N20 Micro DC Geared Motor (120 RPM).

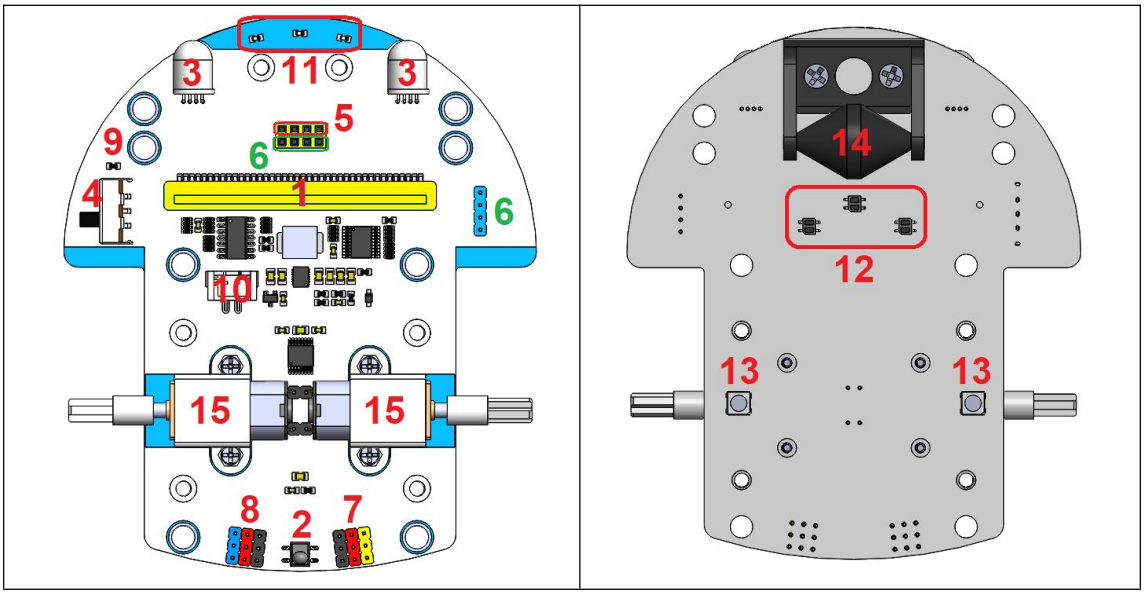
Ultrasonic Type: HC-SR04 (Range: 2cm-400cm, Accuracy: 3mm).

Expansion Port Type: 2.54mm pitch pin headers.

Wheel Diameter: 53mm

Dimensions (L*W*H): 117.5*97*70mm

2.3 Component Description



micro:bit Slot: 1

Insert the micro:bit v2.x.x mainboard here.

Infrared Receiver Sensor: 2

NEC infrared communication protocol, signal pin connected to micro:bit's P9 pin.

Headlight: 3

8mm RGB LED lights, divided into left and right, controlled by STC8H1K08. The micro:bit's I2C port sends relevant commands to the STC8H1K08 to control them.

Power Switch: 4

Controls the power from the battery holder ON/OFF.

Ultrasonic Interface: 5

V=3.3V, G=GND, for connecting external HC-SR04 sonar module. Signal pins connected to micro:bit's P12 (Echo) and P13 (Trig) pins.

I2C Port: 6

The micro:bit's I2C communication interface, SDA=P20, SCL=P19, can communicate with other I2C devices.

Servo Interface: 7

For connecting 3 external servos, used to drive 90-degree, 180-degree, 270-degree, and 360-degree servos.

IO Expansion Port: 8

Can connect other sensor modules.

Power Indicator: 9

Lights steadily during normal operation after battery type is set. Flashes when voltage is less than or equal to the 20% voltage level.

Battery Connector: 10

XH2.54 2P, Input Voltage 3--5.5V.

Infrared Line Tracking Indicator: 11

Lights up when a black line is detected, otherwise turns off.

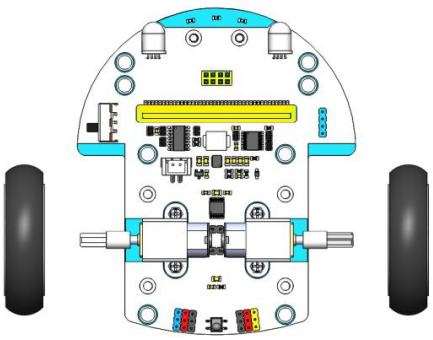
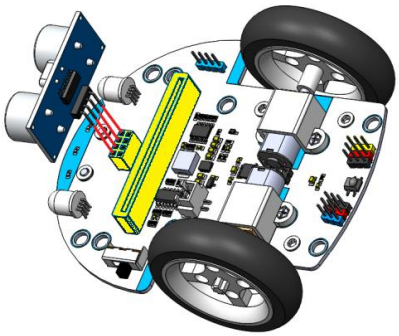
Infrared Line Tracking Sensor: 12

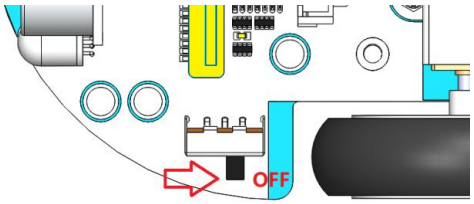
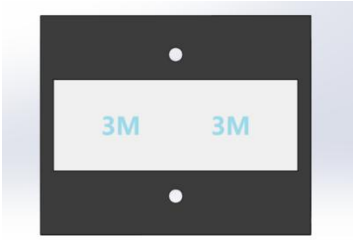
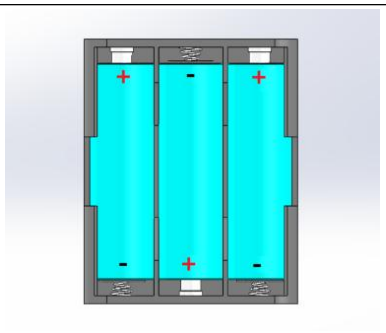
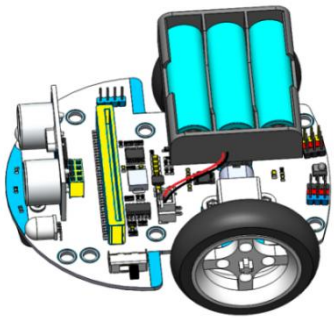
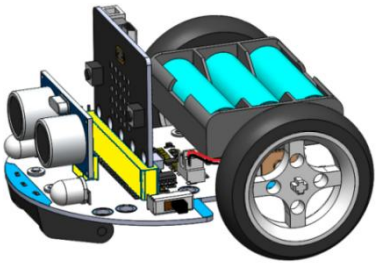
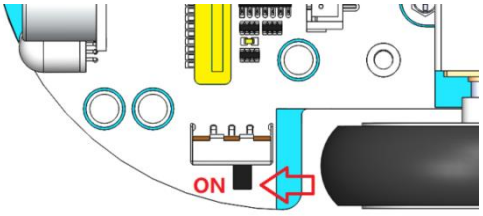
Detects black and white lines via the infrared reflection principle. Divided into left, center, and right three sensors. Signal pins connected to micro:bit's P14, P15, and P16 pins respectively.

RGB Light: 13

WS2812 RGB light, can emit controllable red, green, and blue primary colors. Total of 2 lights, numbered RGB1-RGB2. By controlling the intensity of the three primary colors respectively, various colors of light can be formed. Signal input pin connected to micro:bit's P8 pin.

3. Installing the Pybit

Step 1: Install Wheels	Step 2: Install Ultrasonic Module
	
Step 3: Turn Off Power (Note: Turn off power when inserting or removing micro:bit V2 to	Step 4: Stick 3M Double-Sided Tape to the Center Bottom of the Battery Holder

avoid damage.)	
	
Step 5: Install AA Batteries	Step 6: Stick Battery Holder onto the Car
	
Step 7: Insert micro:bit V2 Board (Note Dot Matrix Faces Forward)	Step 8: Turn On Power Switch
	

Note: It is recommended to turn off the power switch when inserting or removing the micro:bit mainboard to avoid damaging the mainboard!

4. Python Editor Essential Knowledge

The Python Editor is a perfect way to start MicroPython programming with the BBC micro:bit. The Python Editor is free and runs in browsers on all platforms.

We recommend using the Google Chrome or Microsoft Edge browsers. WebUSB is a latest, developing web feature that allows you to access the micro:bit directly from a web page. It also allows you to send data directly from the micro:bit to the Python Editor. It works with Google Chrome and Microsoft Edge browsers.

WebUSB Supports micro:bit Version

If you are not using the latest version of Google Chrome or Microsoft Edge, please ensure they are this version or newer:

Google Chrome (version 79 and above) for Android, Chrome OS, Linux, macOS, and Windows 10.

Microsoft Edge (version 79 and above) for Android, Chrome OS, Linux, macOS, and Windows 10.

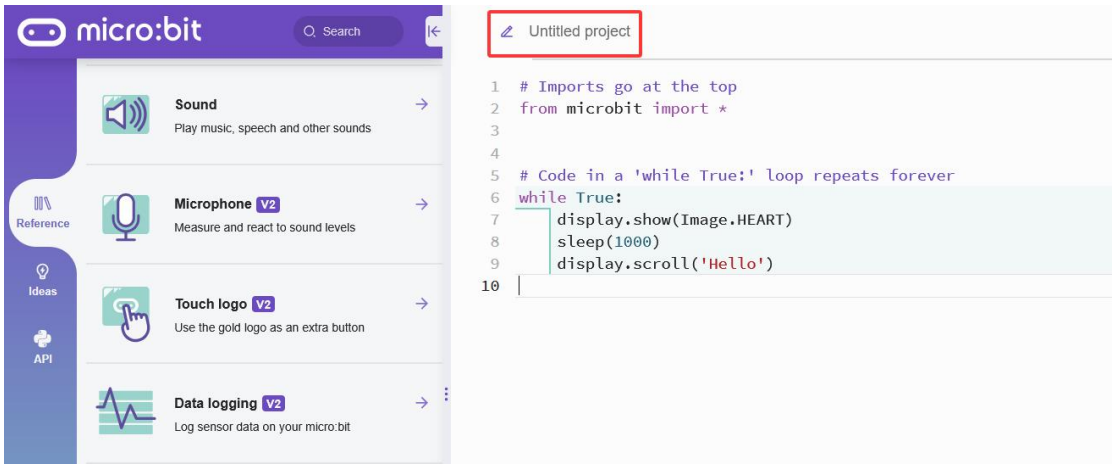
Link to download the latest Google Chrome browser:

https://www.google.com/chrome/

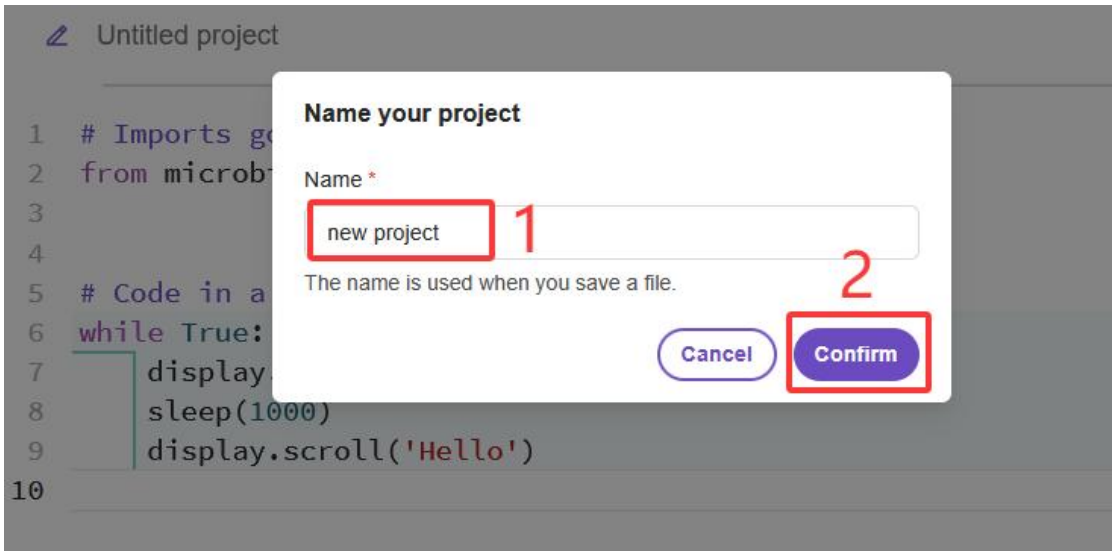
Link to download the latest Microsoft Edge:
https://www.microsoft.com/en-us/edge/download

4.1 Create a New Project

Open the online programming page in PC's Google Chrome browser: https://python.microbit.org/v/3, click the red box:

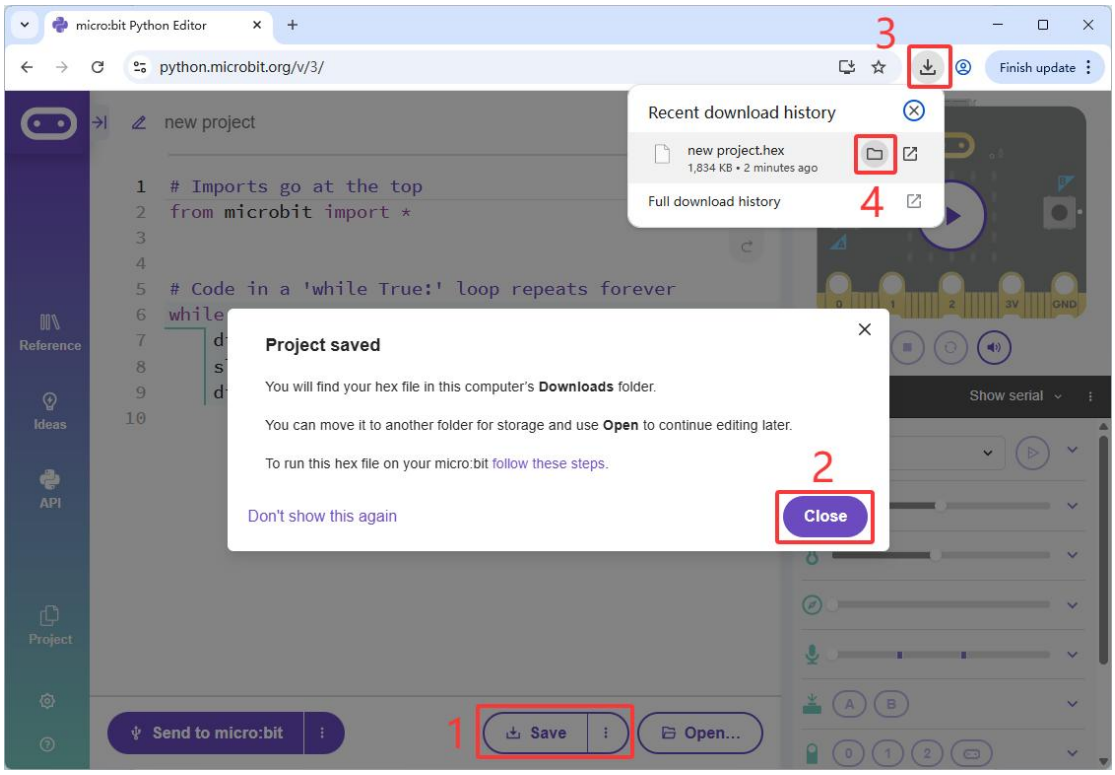


Then enter the project name and click "Confirm":



4.2 Save Project Locally

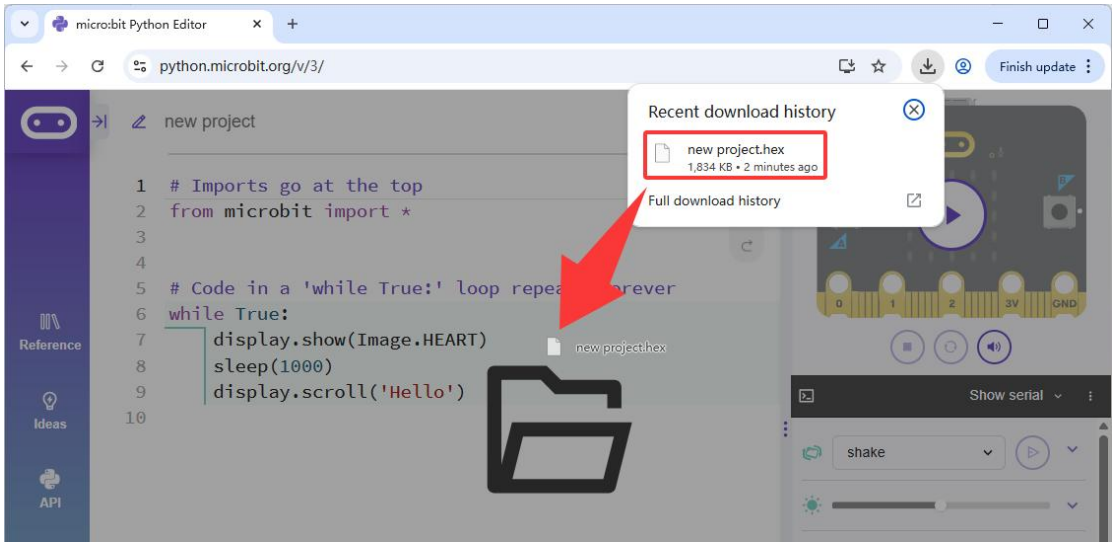
Click step 1 "Save" button, then click step 2 "Close" button to save the project locally. Then follow steps 3 and 4 to find the saved ".hex" file, as shown below:



Note: If you don't want the pop-up window to appear again, click "Don't show this again" in step 2!

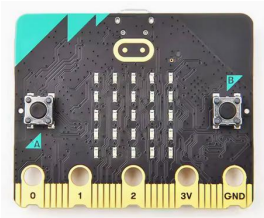
4.3 Import Local Project

Directly drag and drop the local "HEX" project file into the working area of the Python Editor, as shown below:

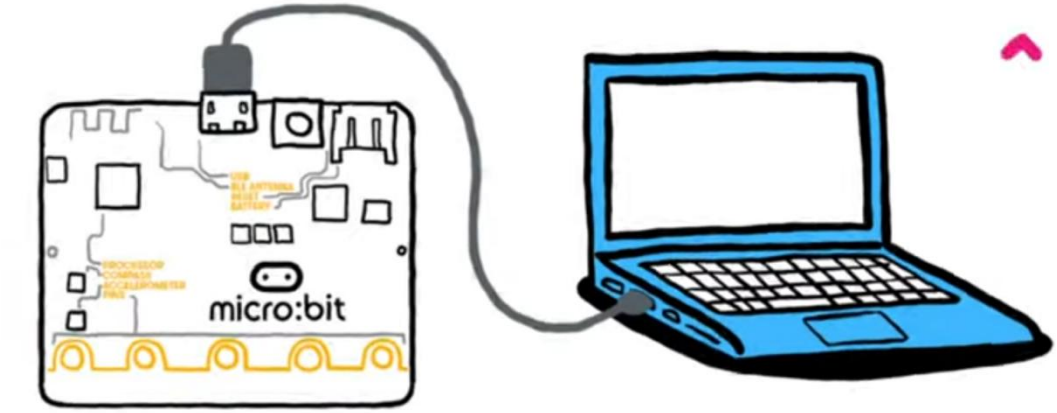


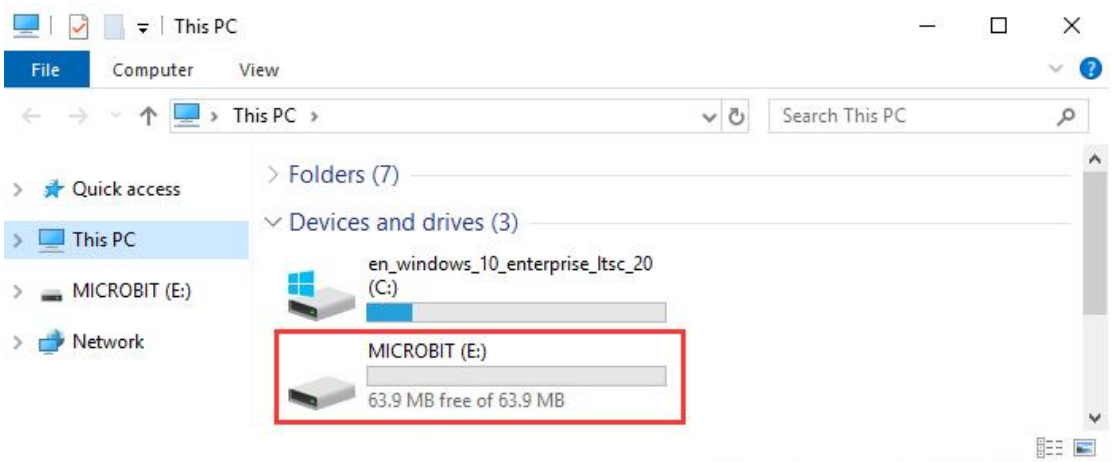
4.4 Upload Project

Tools:

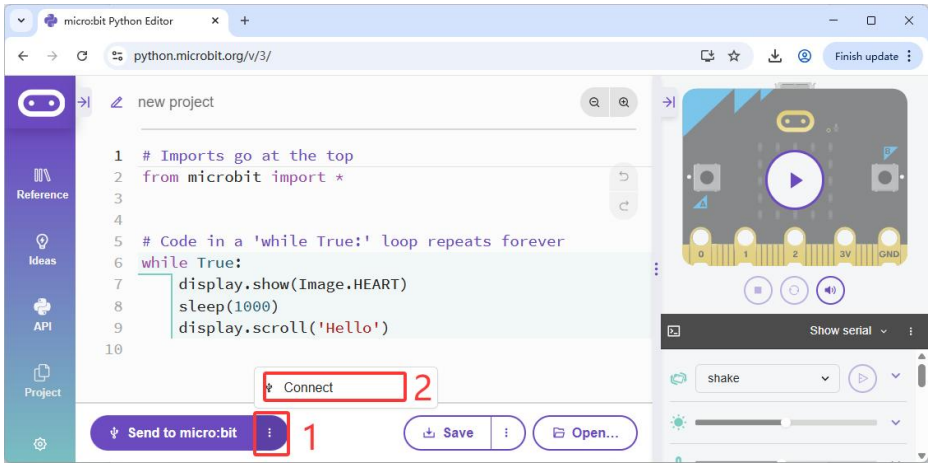
PC	micro:bit v2.x.x	Micro USB Cable
		

Connect the micro:bit mainboard to the PC using a Micro USB cable. A storage drive will appear on the PC:

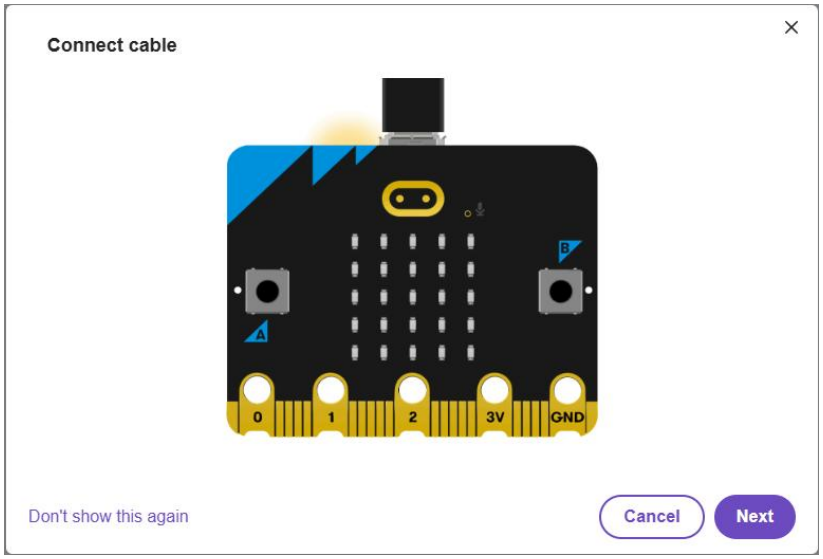




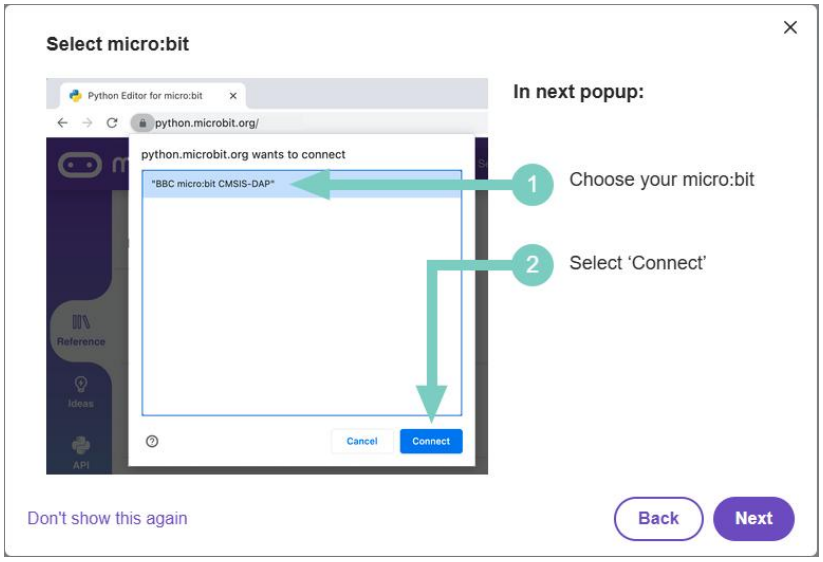
Click the "Connect" button:



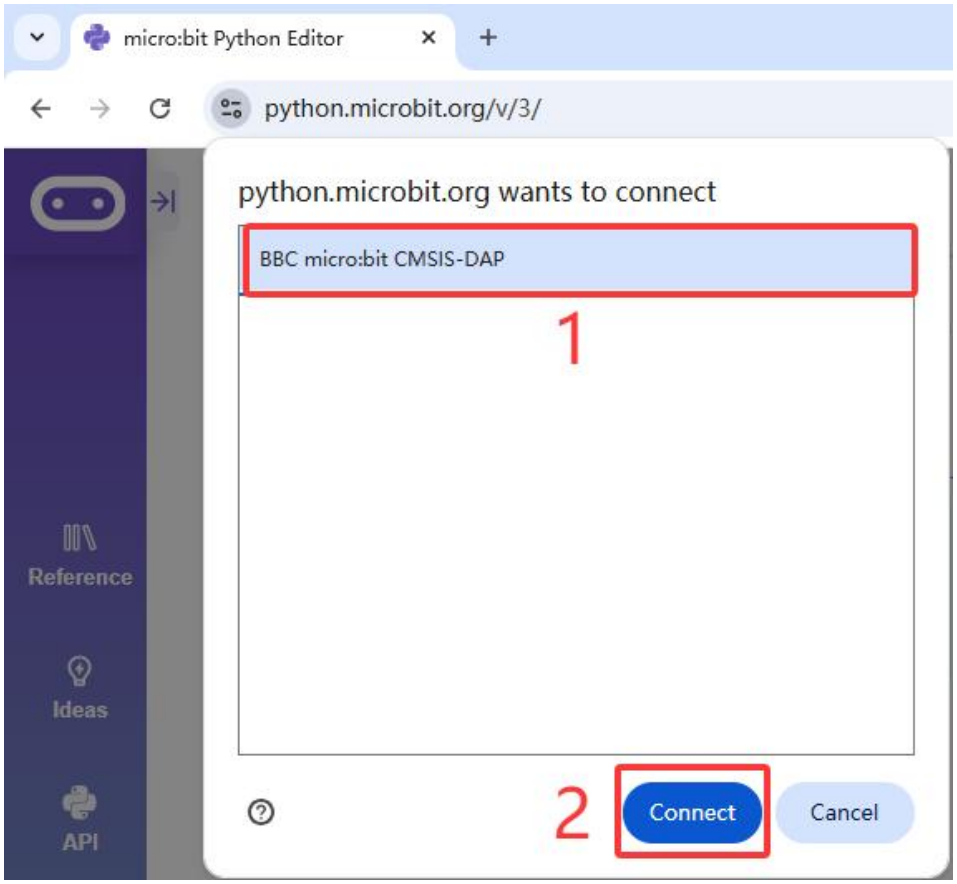
Click "Next":



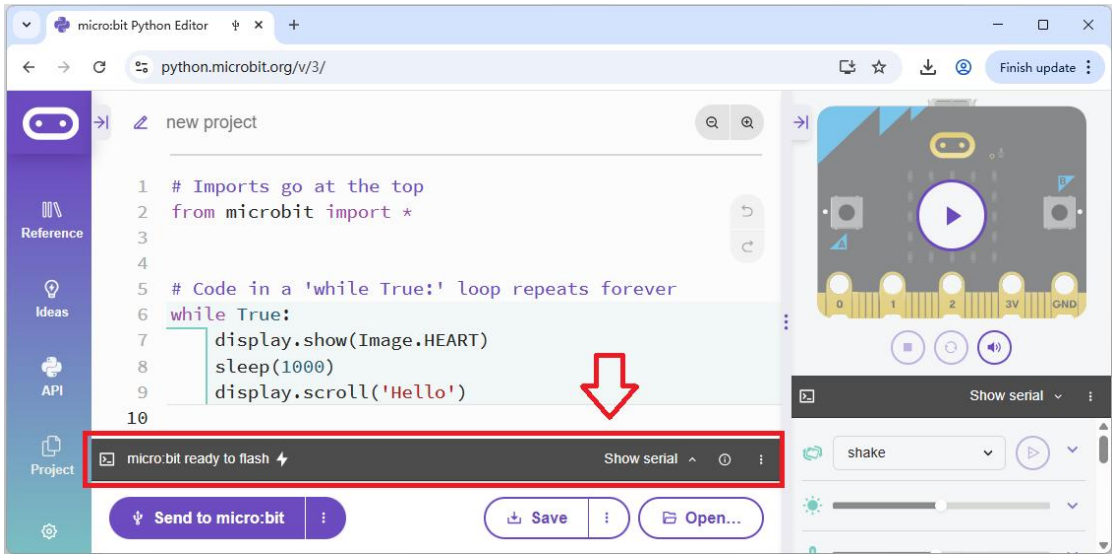
Click "Next":



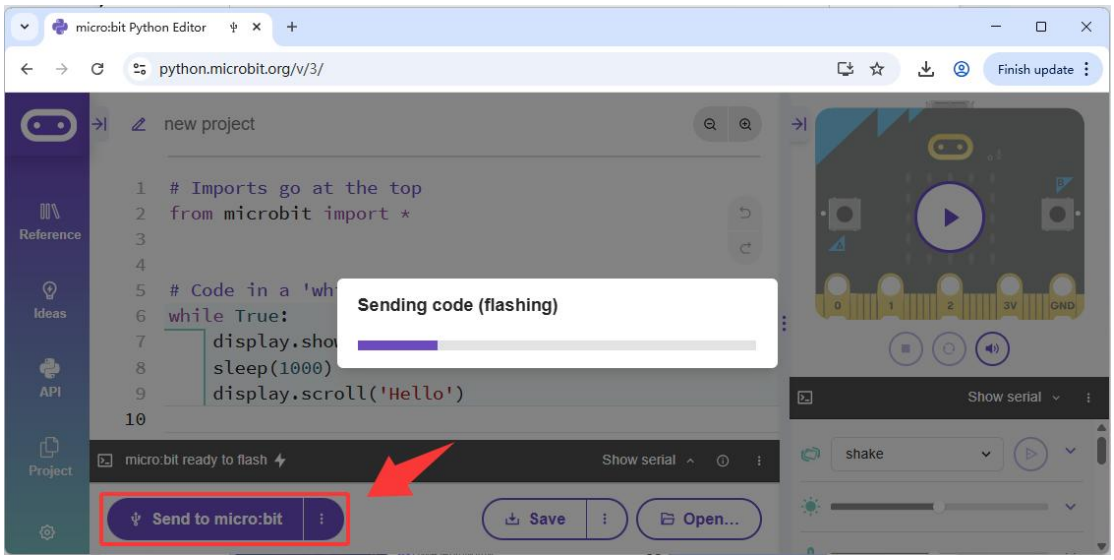
Then follow these steps:



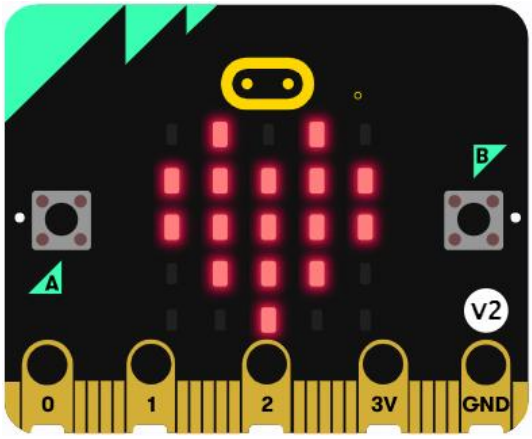
When the following appears, it indicates a successful connection between the micro:bit and the Python Editor:



Now click "Send to micro:bit" to upload the code from the Python Editor to the micro:bit:

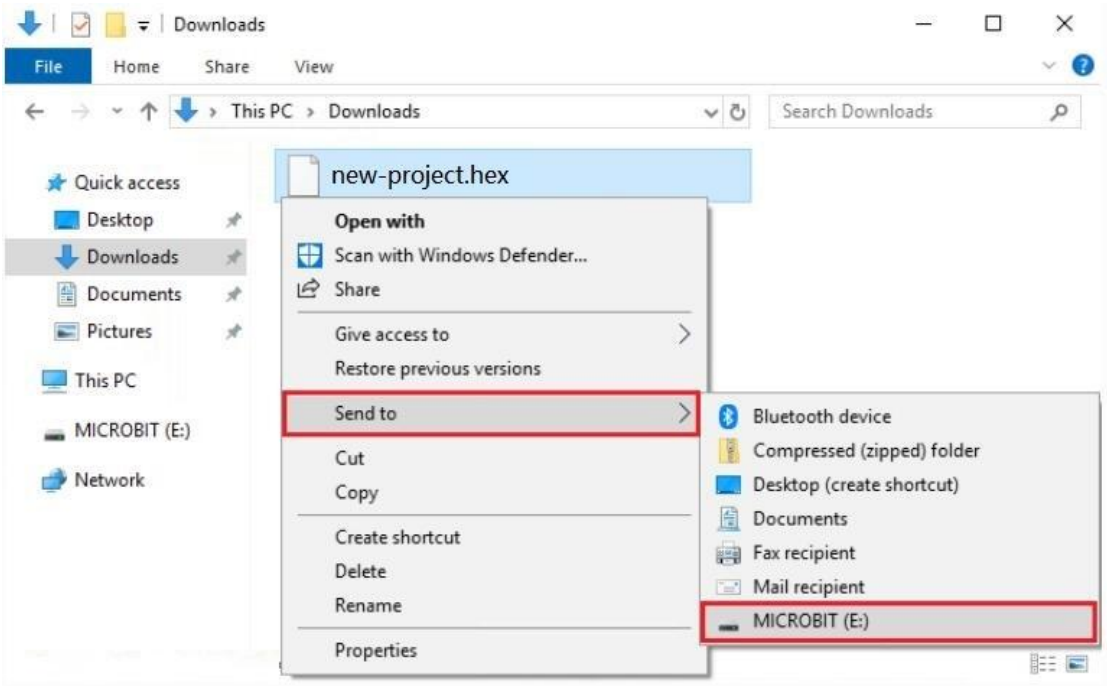


After the code is uploaded, the micro:bit mainboard's dot matrix displays a heart shape, then scrolls "Hello":

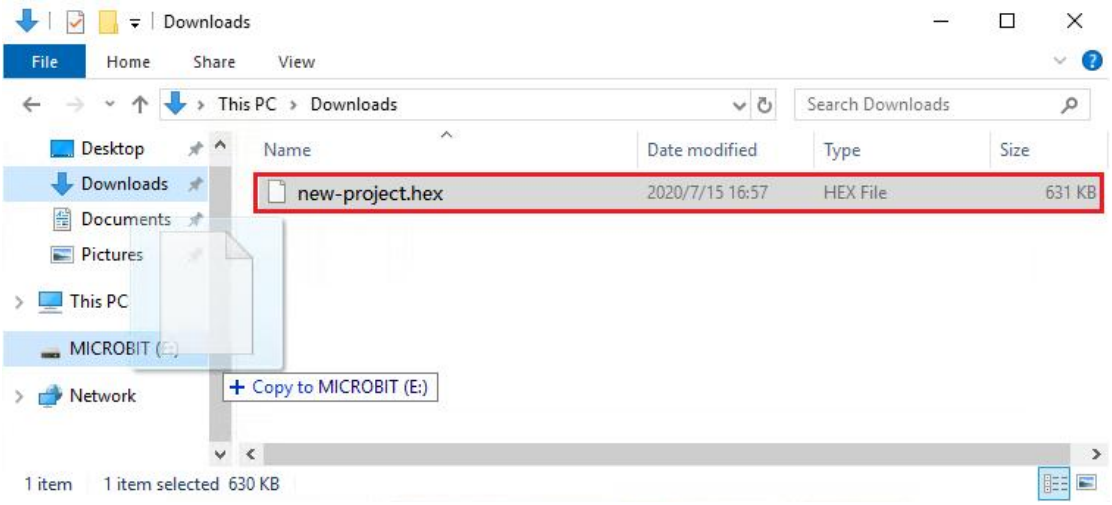


2 Other Methods to Upload Code

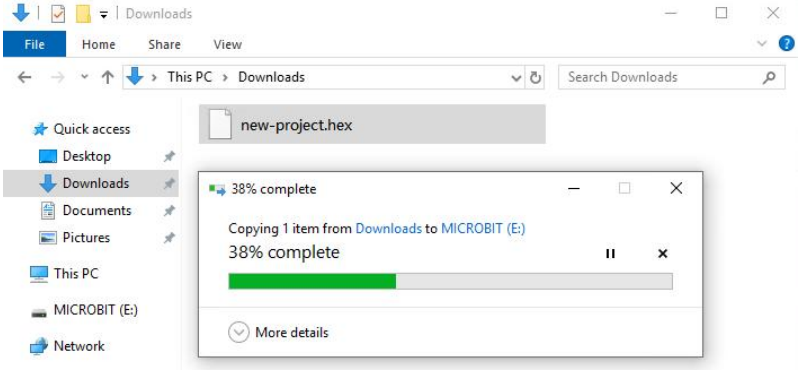
Select the "HEX" file, right-click, and send the "HEX" file to the micro:bit mainboard:



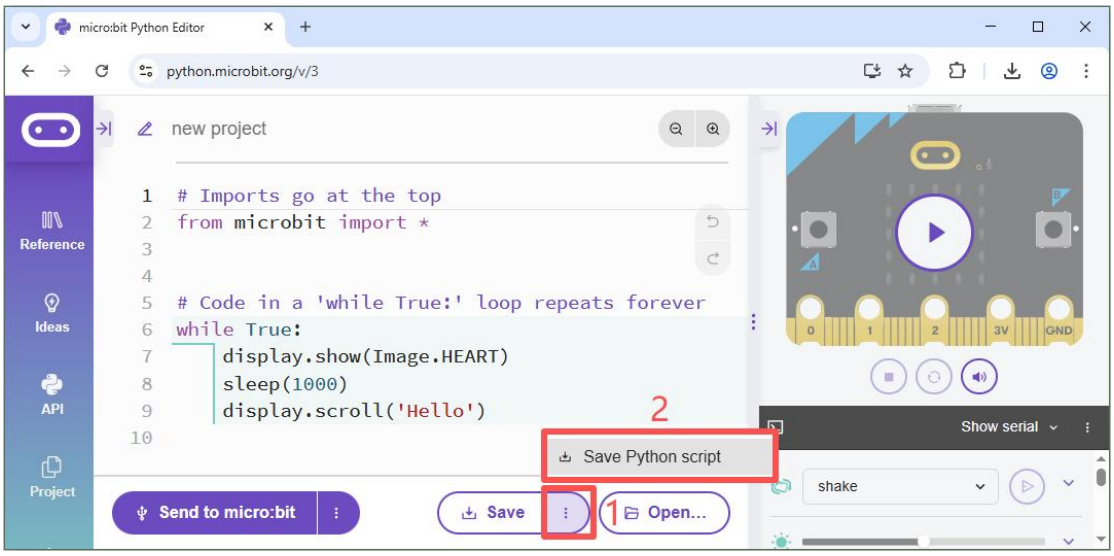
Or drag and drop it directly onto the micro:bit mainboard:

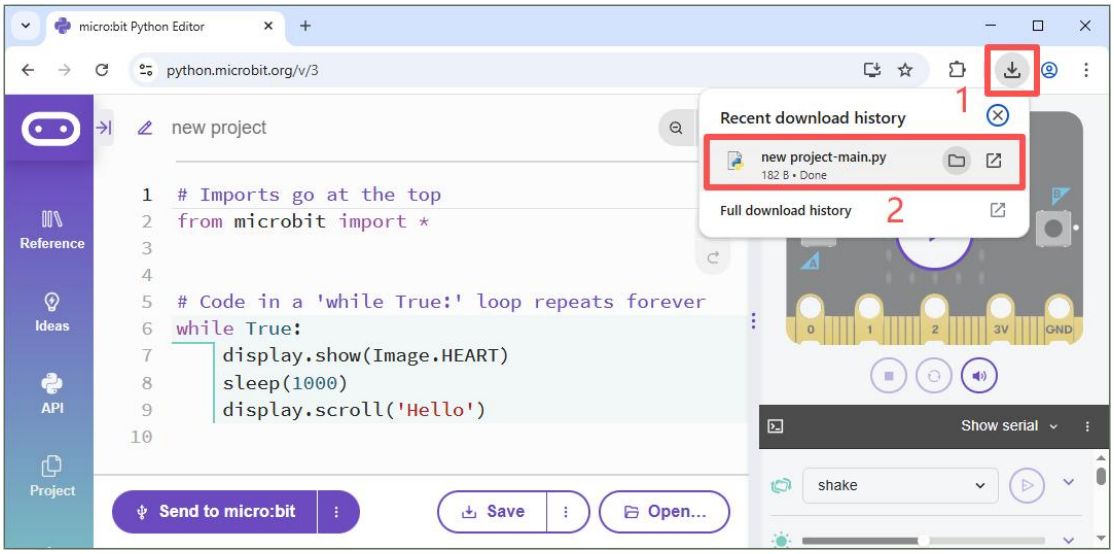


The following interface indicates code is being uploaded:



4.5 Save Project as Python File





Note: Python "xx.py" files cannot run directly on the micro:bit; only the project's "xx.hex" file can run on the micro:bit. "xx.py" files can be opened with general text editing software, while "xx.hex" files cannot!

4.6 Python Editor Basic Syntax Learning

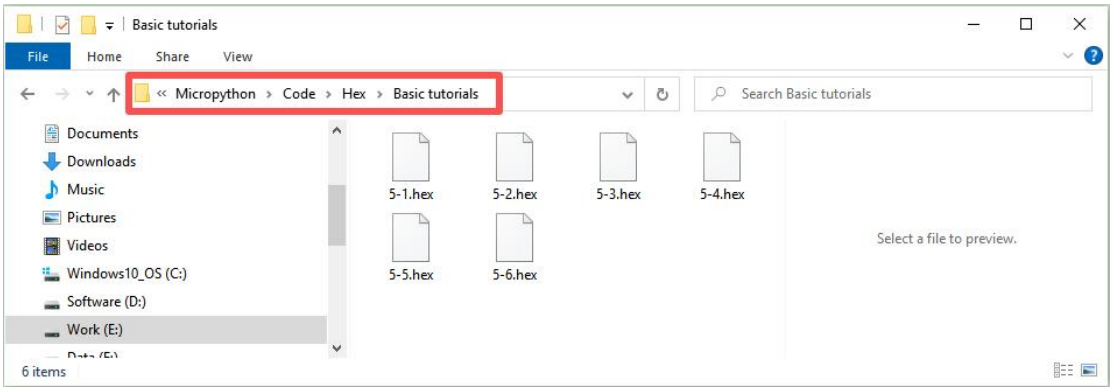
The micro:bit platform provides official Python Editor usage reference documentation.

User Guide: https://microbit.org/get-started/user-guide/python-editor/

MicroPython Documentation: https://microbit-micropython.readthedocs.io/en/v2-docs/

5. Pybit Basic Tutorials

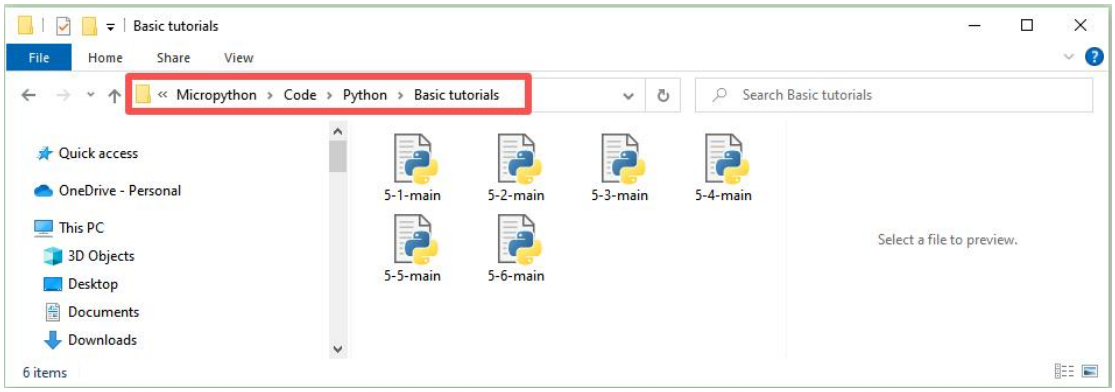
The example projects for the basic tutorials are all saved in the "Micropython -> Code -> Hex -> Basic tutorials" folder:



Special Reminder!!!

For the following tutorials, if you are importing our provided local example projects, please refer to section 4.3. If you are

The example Python files for the basic tutorials are saved in the "Micropython -> Code -> Python -> Basic tutorials" folder:

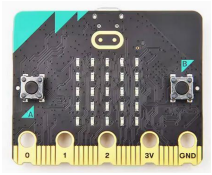
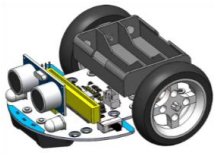


5.1 Battery Fuel Gauge

Objective

Set the battery type and read the battery voltage level.

Tools

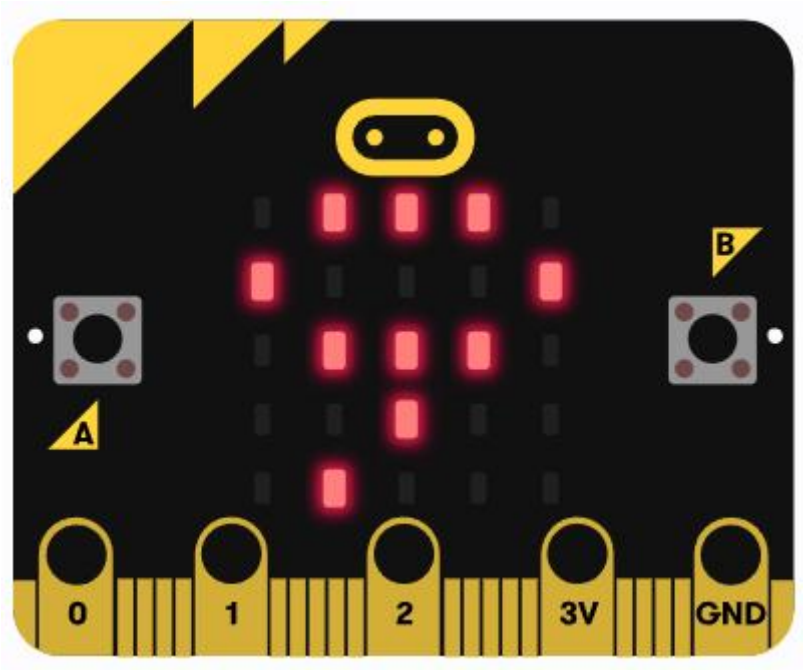
Computer	micro:bit v2.x.x	Micro USB Cable	Pybit
			

Programming

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# Type of battery
aaBattery = bytearray([0x01])
lithiumBattery = bytearray([0x02])

# Initialize the I2C communication interface
i2c.init()


# Code in a 'while True:' loop repeats forever
while True:
    # Read the battery power level
    i2c.write(i2cAddr, aaBattery, True)
    batLevel = i2c.read(i2cAddr, 1)

    sleep(1000)                 # Delay: 1000 milliseconds
    display.scroll(batLevel[0])  # Scroll display of battery level
    sleep(3000)                 # Delay: 3000 milliseconds
```

Conclusion

The micro:bit's dot matrix scrolls to display the read voltage level, value range: 0--100%. And when the battery voltage level is less than or equal to 20%, the battery indicator light stays on steadily; otherwise, it flashes.



Thinking

How to make the micro:bit's buzzer sound an alarm when the voltage level is less than 20%?

Common Issues

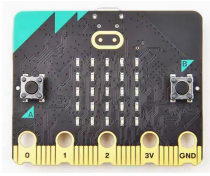
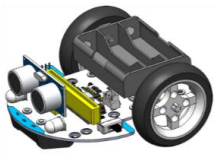
The statement to read the battery voltage level must be executed at least once in the program to set the battery type, otherwise the battery voltage indicator cannot work properly!

5.2 Headlights

Objective

Make the Pybit's headlights flash.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit
			

Programming

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For head RGB LED
leftLed  = 0
rightLed = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the head RGB LED function
# led: 0 is the left led and 1 is the right LED.
# r: red brightness value
# g: green brightness value
# b: blue brightness value
def setHeadRgbLed(led, r, g, b):
    if led == leftLed:
        i2cBuf[0] = 0x07  # red register
        i2cBuf[1] = r     # red brightness value
        i2c.write(i2cAddr, i2cBuf)
        i2cBuf[0] = 0x08  # green register
        i2cBuf[1] = g     # green brightness value
        i2c.write(i2cAddr, i2cBuf)
        i2cBuf[0] = 0x09  # blue register
        i2cBuf[1] = b     # blue brightness value
        i2c.write(i2cAddr, i2cBuf)
```

```
    if led == rightLed:
        i2cBuf[0] = 0x0a  # red register
        i2cBuf[1] = r     # red brightness value
        i2c.write(i2cAddr, i2cBuf)
        i2cBuf[0] = 0x0b  # green register
        i2cBuf[1] = g     # green brightness value
        i2c.write(i2cAddr, i2cBuf)
        i2cBuf[0] = 0x0c  # blue register
        i2cBuf[1] = b     # blue brightness value
        i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    setHeadRgbLed(leftLed, 255, 0, 0)    # red
    setHeadRgbLed(rightLed, 255, 0, 0)
    sleep(1000)
    setHeadRgbLed(leftLed, 0, 255, 0)    # green
    setHeadRgbLed(rightLed, 0, 255, 0)
    sleep(1000)
    setHeadRgbLed(leftLed, 0, 0, 255)    # blue
    setHeadRgbLed(rightLed, 0, 0, 255)
    sleep(1000)
```

Conclusion

The Pybit's 2 headlights cycle through colors: Red -> Delay 1 second -> Green -> Delay 1 second -> Blue -> Delay 1 second.

Thinking

How to make the headlights display more colors faster?

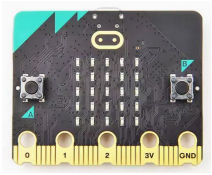
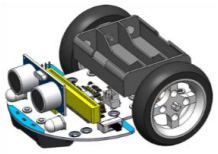
Common Issues

5.3 RGB Light

Objective

Make the two RGB lights at the bottom of the Pybit flash.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit
			

Programming

```
# micro:bit V2 only
# Imports go at the top
from microbit import *
import neopixel

np = neopixel.NeoPixel(pin8, 2)  # Using pin8, 2 neopixel.
# Code in a 'while True:' loop repeats forever
while True:
    np.fill((255, 0, 0))  # Light all NeoPixels red
    np.show()
    sleep(1000)
    np.fill((0, 255, 0))  # Light all NeoPixels green
    np.show()
    sleep(1000)
    np.fill((0, 0, 255))  # Light all NeoPixels blue
    np.show()
    sleep(1000)
    np.clear()            # Clear NeoPixels

    np[0] = (255, 0, 0)   # A NeoPixel bright red
    np[1] = (0, 255, 0)   # The other NeoPixel is bright green
    np.show()
    sleep(1000)
    np.clear()            # Clear NeoPixels
```

Conclusion

The 2 rainbow lights at the bottom of the Pybit cycle through colors: Red -> Delay 1 second -> Green -> Delay 1 second -> Blue -> Delay 1 second.

Thinking

How to make the rainbow lights display more colors faster?

Common Issues

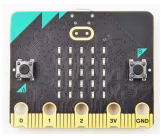
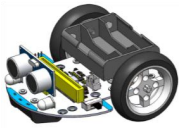
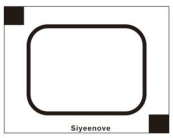
This code can only run on micro:bit V2; micro:bit V1 will report an error!

5.4 Line Tracking

Objective

Use the Pybit's 3 infrared line tracking sensors to identify black and white.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit	Map
				

Programming

```
# Imports go at the top
from microbit import *

channel1 = 0   # left   = P14
channel2 = 0   # centre = P15
channel3 = 0   # right  = P16

pin14.set_pull(pin14.PULL_UP)
pin15.set_pull(pin15.PULL_UP)
pin16.set_pull(pin16.PULL_UP)
```

```
# Code in a 'while True:' loop repeats forever
while True:
    # Read the values of three sensors, 0 or 1.
    # 0: Black is detected.
    # 1: White is detected.
    channel1 = pin14.read_digital()
    channel2 = pin15.read_digital()
    channel3 = pin16.read_digital()

    # Convert the values of the three sensors into one data.
    # Binary data notation:
    # 0b bit2 bit1 bit0
    # channel1 = bit2, chanel2 = bit1, chanel3 = bit0.
    value = (channel1 << 2) + (channel2 << 1) + channel3

    if value == 0b111:
        display.show(Image.NO)
    if value == 0b000:
        display.show(Image.YES)
    sleep(250)
```

Conclusion

Rough black surfaces absorb infrared light well. When the infrared light emitted by the sensor hits a black line, it is absorbed; when it hits a white object, the infrared signal is reflected back.

After uploading the code, place all 3 of the Pybit's infrared reflection sensors on the black area of the map, the micro:bit dot matrix displays "√"; if all 3 infrared reflection sensors are placed on the white area of the map, the micro:bit dot matrix displays "×".

Thinking

How to distinguish which infrared reflection sensor detected the black object?

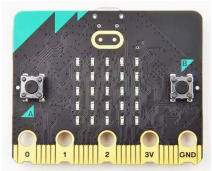
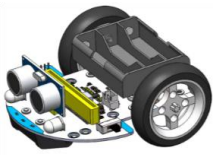
Common Issues

5.5 Ultrasonic Sensor

Objectives

Use ultrasonic to measure the distance to an obstacle.

Tools

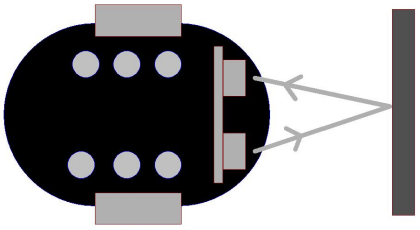
PC	micro:bit v2.x.x	Micro USB Cable	Pybit
			

Programming

```
# Imports go at the top
from microbit import *
import utime

# Define the pins of the ultrasonic module.
TRIG_PIN = pin13
ECHO_PIN = pin12

# Initialize the ultrasonic module
def setup_ultrasonic():
    TRIG_PIN.write_digital(0)
    ECHO_PIN.set_pull(ECHO_PIN.NO_PULL)

# Measuring distance (cm)
def measure_distance():
    # A trigger signal of 10μs was sent
    TRIG_PIN.write_digital(1)
    utime.sleep_us(10)
    TRIG_PIN.write_digital(0)

    # Wait for the echo pin to get high
    timeout = 50000   # 50ms timeout
    start_time = utime.ticks_us()
    while ECHO_PIN.read_digital() == 0:
```

```
        if utime.ticks_diff(utime.ticks_us(), start_time) > timeout:
            return -1  # timeout

    # The echo onset time was recorded
    pulse_start = utime.ticks_us()

    # Wait for the echo pin to get low
    while ECHO_PIN.read_digital() == 1:
        if utime.ticks_diff(utime.ticks_us(), pulse_start) > timeout:
            return -1  # timeout

    # The echo end time was recorded
    pulse_end = utime.ticks_us()

    # Calculate the pulse duration
    pulse_duration = utime.ticks_diff(pulse_end, pulse_start)

    # Calculating distance（The Speed of Sound
    # 340m/s = 0.034cm/μs，Round trip，so divided by 2）
    distance = (pulse_duration * 0.034) / 2

    return distance

setup_ultrasonic()

# Code in a 'while True:' loop repeats forever
while True:
    # measuring distance
    dist = measure_distance()

    if dist == -1:    # Measurement timeout
        display.show(Image.NO)
    elif dist > 200:  # Greater than the maximum measurement range
        display.show(Image.ASLEEP)
    else:
        display.scroll(str(int(dist)))   # Displaying distance values

    sleep(1000)
```

Conclusion



The micro:bit's dot matrix scrolls to display the measured obstacle distance in centimeters.

Thinking

Common Issues

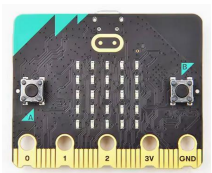
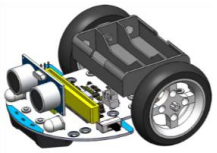
It is best to have a 50ms delay between two ultrasonic distance measurements to prevent interference between the two measurement beams.

5.6 Wheel Rotation

Objective

Learn how to drive the Pybit's 2 motors.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit
			

Programming

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
```

```
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    # CW
    setWheelSpeed(leftwheel, backward, 100)
    setWheelSpeed(rightwheel, forward, 100)
    sleep(1000)
    # stop
    setWheelSpeed(leftwheel, backward, 0)
    setWheelSpeed(rightwheel, forward, 0)
    sleep(1000)
    # CCW
    setWheelSpeed(leftwheel, forward, 100)
    setWheelSpeed(rightwheel, backward, 100)
    sleep(1000)
    # stop
    setWheelSpeed(leftwheel, forward, 0)
    setWheelSpeed(rightwheel, backward, 0)
    sleep(1000)
```

Conclusion

The Pybit is rotating left and right in place.

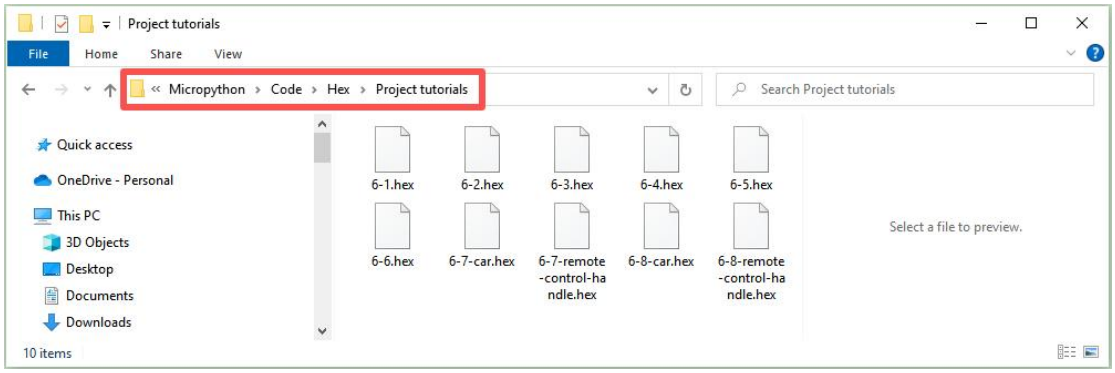
Thinking

How to make the Pybit move forward?

Common Issues

6. Pybit Project Tutorials

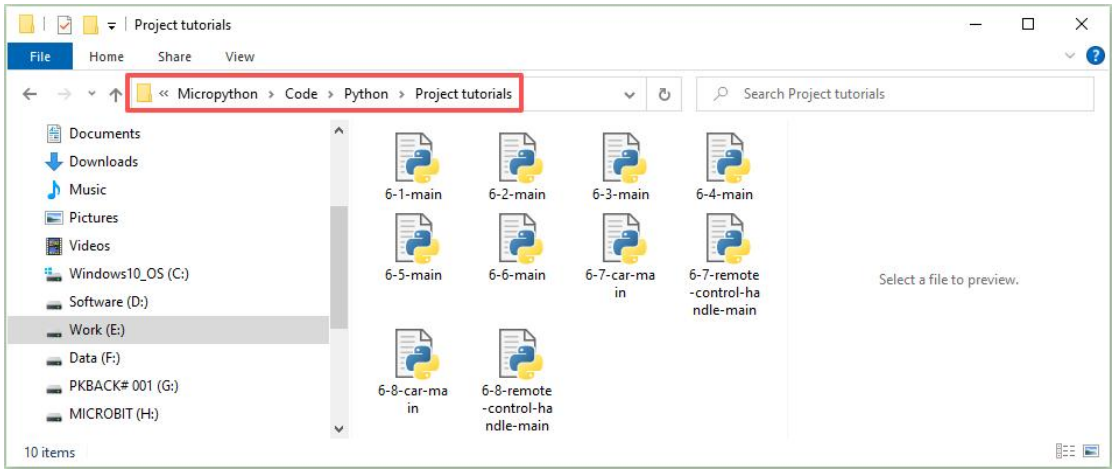
The example projects for the project tutorials are all saved in the "Micropython -> Code -> Hex -> Project tutorials" folder:



Special Reminder!!!

For the following tutorials, if you are importing our provided local example projects, please refer to section 4.3. If you are creating a new project, please refer to section 4.1.

The example Python files for the project tutorials are saved in the "Micropython -> Code -> Python -> Project tutorials" folder:



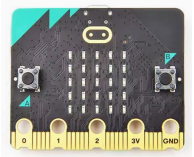
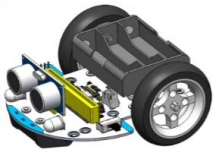
6.1 Uniform Acceleration Straight Line

Objective

Make the Pybit move forward in a straight line with uniform acceleration.

Note: The Pybit motor is very powerful. If the initial acceleration is too high, the front caster wheel may lift off the ground.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit
			

Programming

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
```

```
        speed = 0
    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    if speed > 100:
        speed = 0
    # Run forward
    setWheelSpeed(leftwheel, forward, speed)
    setWheelSpeed(rightwheel, forward, speed)

    sleep(30)
    speed = speed + 1
```

Conclusion

The car starts moving at a constant speed in a cycle, preventing the front wheels from lifting off the ground due to excessive speed.

Thinking

How to program the car to accelerate uniformly and then decelerate uniformly?

Common issues

If the compensation value is not set at the factory, it may cause the motor to not rotate. Simply reset the compensation value to resolve this!

The car can internally compensate for the speed of the two wheels, allowing it to move in a relatively straight line. This compensation value is permanently stored in the car's EEPROM. The default factory compensation value is 0. You can use the following code to reset the compensation value:

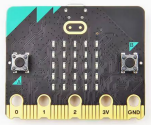
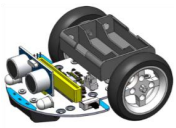
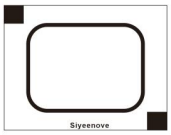
```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a
i2cBuf = bytearray([0x00, 0x00])

# For wheel
leftwheel  = 0
rightwheel = 1

# Calibrate the wheel speed function
# leftWheelOffset:  0--10
# rightWheelOffset: 0--10
def wheelsAdjustment(leftWheelOffset, rightWheelOffset):
    # The limit variable is between 0 and 10.
    leftWheelOffset = max(0, min(10, leftWheelOffset))
    rightWheelOffset = max(0, min(10, rightWheelOffset))

    i2cBuf[0] = 0x03  # Left wheel offset register
    i2cBuf[1] = leftWheelOffset
    i2c.write(i2cAddr, i2cBuf)
    i2cBuf[0] = 0x04  # Right wheel offset register
    i2cBuf[1] = rightWheelOffset
    i2c.write(i2cAddr, i2cBuf)

wheelsAdjustment(0, 0)
while True:
    None
```

6.2 Follow Black Line

Objective

Make the Pybit follow the black line and run in circles.

Tools

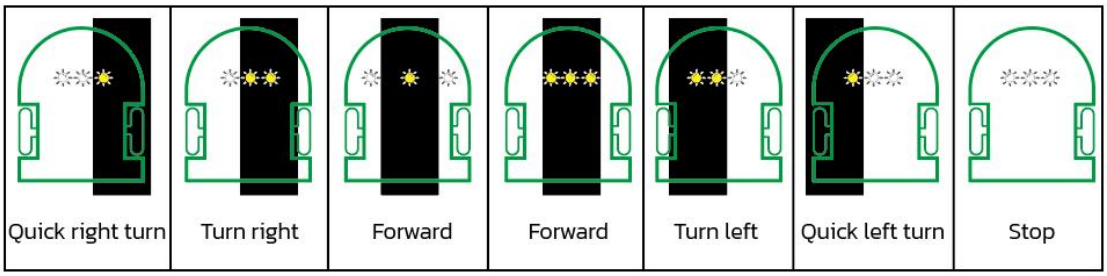
PC	micro:bit v2.x.x	Micro USB Cable	Pybit	map
				

Programming

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# For line Tracking Sensor
channel1 = 0   # left   = P14
channel2 = 0   # centre = P15
channel3 = 0   # right  = P16
pin14.set_pull(pin14.PULL_UP)
pin15.set_pull(pin15.PULL_UP)
pin16.set_pull(pin16.PULL_UP)

display.show(Image.HAPPY)
sleep(400)

# Set the wheel speed function
```

```
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    # Read the values of three sensors, 0 or 1.
    # 0: Black is detected.
    # 1: White is detected.
    channel1 = pin14.read_digital()
    channel2 = pin15.read_digital()
    channel3 = pin16.read_digital()

    # Convert the values of the three sensors into one data.
    # Binary data notation:
    # 0b bit2 bit1 bit0
    # channel1 = bit2, chanel2 = bit1, chanel3 = bit0.
    value = (channel1 << 2) + (channel2 << 1) + channel3
```

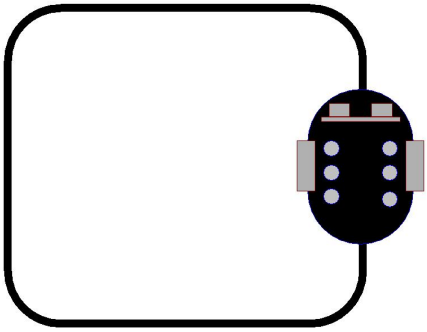
```
if value == 0b100:  # Slow right turn
    setWheelSpeed(leftwheel, forward, 25)
    setWheelSpeed(rightwheel, forward, 0)

if value == 0b001:  # Slow left turn
    setWheelSpeed(leftwheel, forward, 0)
    setWheelSpeed(rightwheel, forward, 25)

if value == 0b110:  # Quick right turn
    setWheelSpeed(leftwheel, forward, 40)
    setWheelSpeed(rightwheel, forward, 0)

if value == 0b011:  # Quick left turn
    setWheelSpeed(leftwheel, forward, 0)
    setWheelSpeed(rightwheel, forward, 40)

if value == 0b101 or value == 0b000:   # Forward
    setWheelSpeed(leftwheel, forward, 20)
    setWheelSpeed(rightwheel, forward, 20)

if value == 0b111:  # Stop
    setWheelSpeed(leftwheel, forward, 0)
    setWheelSpeed(rightwheel, forward, 0)
```

						
Quick right turn	Turn right	Forward	Forward	Turn left	Quick left turn	Stop
0b110	0b100	0b101	0b000	0b001	0b011	0b111

 indicates no line detected (1).

 indicates black line detected (0).

Conclusion



The Pybit will follow the black line on the map at a constant speed.

Thinking

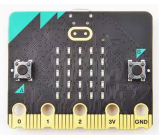
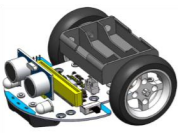
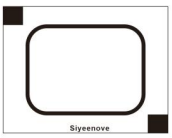
Common Issues

6.3 Perimeter Patrol

Objective

Make the Pybit run within the black line frame on the map.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit	Map
				

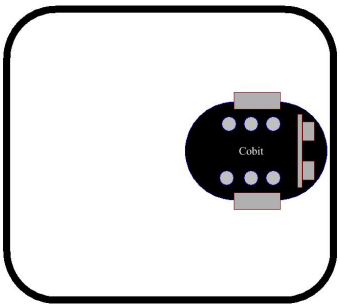
Programming

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# For line Tracking Sensor
channel1 = 0   # left   = P14
channel2 = 0   # centre = P15
channel3 = 0   # right  = P16
pin14.set_pull(pin14.PULL_UP)
pin15.set_pull(pin15.PULL_UP)
pin16.set_pull(pin16.PULL_UP)

display.show(Image.HAPPY)
sleep(400)

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0
    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
```

```
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    # Read the values of three sensors, 0 or 1.
    # 0: Black is detected.
    # 1: White is detected.
    channel1 = pin14.read_digital()
    channel2 = pin15.read_digital()
    channel3 = pin16.read_digital()

    # Convert the values of the three sensors into one data.
    # Binary data notation:
    # 0b bit2 bit1 bit0
    # channel1 = bit2, chanel2 = bit1, chanel3 = bit0.
    value = (channel1 << 2) + (channel2 << 1) + channel3

    if value == 0b111:   # Forward
        setWheelSpeed(leftwheel, forward, 40)
        setWheelSpeed(rightwheel, forward, 40)
    else:
        # Stop
        setWheelSpeed(leftwheel, forward, 0)
        setWheelSpeed(rightwheel, forward, 0)
        # Backward
        setWheelSpeed(leftwheel, backward, 40)
        setWheelSpeed(rightwheel, backward, 40)
```

```
        sleep(200)

        if random.randint(0, 1) == 0:
            # Left turn
            setWheelSpeed(leftwheel, forward, 0)
            setWheelSpeed(rightwheel, backward, 40)
            sleep(300)
        else:
            # Right turn
            setWheelSpeed(leftwheel, backward, 40)
            setWheelSpeed(rightwheel, forward, 0)
            sleep(300)
```

Conclusion



The Pybit will keep running within the black line frame.

Thinking

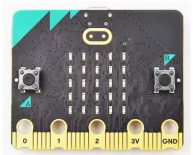
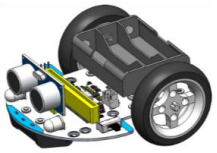
Common Issues

6.4 Fixed Distance Following

Objective

Make your Pybit dynamically maintain a fixed distance from your hand.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit
			

Programming

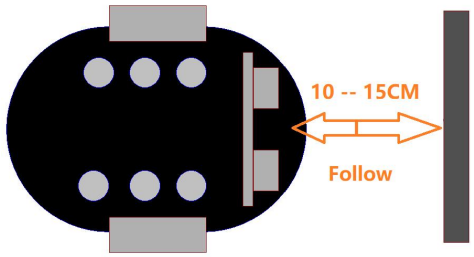
```
# Imports go at the top
from microbit import *
import utime

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# Define the pins of the ultrasonic module.
TRIG_PIN = pin13
ECHO_PIN = pin12

# Initialize the ultrasonic module
def setup_ultrasonic():
    TRIG_PIN.write_digital(0)
    ECHO_PIN.set_pull(ECHO_PIN.NO_PULL)
# Measuring distance (cm)
def measure_distance():
```

```
    # A trigger signal of 10μs was sent
    TRIG_PIN.write_digital(1)
    utime.sleep_us(10)
    TRIG_PIN.write_digital(0)

    # Wait for the echo pin to get high
    timeout = 50000   # 50ms timeout
    start_time = utime.ticks_us()
    while ECHO_PIN.read_digital() == 0:
        if utime.ticks_diff(utime.ticks_us(), start_time) > timeout:
            return -1  # timeout

    # The echo onset time was recorded
    pulse_start = utime.ticks_us()

    # Wait for the echo pin to get low
    while ECHO_PIN.read_digital() == 1:
        if utime.ticks_diff(utime.ticks_us(), pulse_start) > timeout:
            return -1  # timeout

    # The echo end time was recorded
    pulse_end = utime.ticks_us()

    # Calculate the pulse duration
    pulse_duration = utime.ticks_diff(pulse_end, pulse_start)

    # Calculating distance（The Speed of Sound
    # 340m/s = 0.034cm/μs，Round trip, so divided by 2）
    distance = (pulse_duration * 0.034) / 2

    return distance

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
```

```
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

setup_ultrasonic()
display.show(Image.HAPPY)
sleep(400)

# Code in a 'while True:' loop repeats forever
while True:
    # measuring distance
    dist = measure_distance()

    if dist > 10 and dist < 15:
        # Stop
        setWheelSpeed(leftwheel, forward, 0)
        setWheelSpeed(rightwheel, forward, 0)
    elif dist < 10:
        # Backward
        setWheelSpeed(leftwheel, backward, 50)
        setWheelSpeed(rightwheel, backward, 50)
```

```
else:
    # Forward
    setWheelSpeed(leftwheel, forward, 50)
    setWheelSpeed(rightwheel, forward, 50)
```

Conclusion



When the car is too far from the hand, it moves forward to get closer; when too close, it moves away; when the distance is appropriate, it stops.

Thinking

Common Issues

Q: When using the Pybit, it was found that the car was running normally initially, but after connecting the ultrasonic sensor, the car malfunctioned and could not drive.

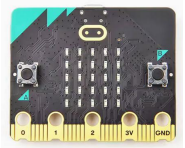
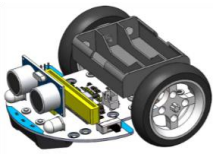
A: Please check if the ultrasonic sensor is plugged into the wrong interface; it should be plugged into the sonar interface, not the I2C interface.

6.5 Ultrasonic Obstacle Avoidance

Objective

Make your Pybit automatically avoid obstacles while driving.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit
			

Programming

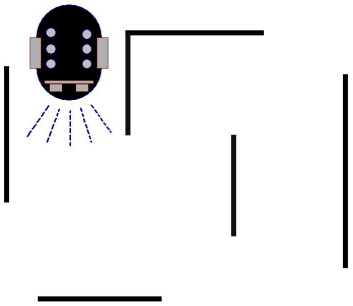
```
# Imports go at the top
from microbit import *
import utime
import random

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# Define the pins of the ultrasonic module.
TRIG_PIN = pin13
ECHO_PIN = pin12

# Initialize the ultrasonic module
def setup_ultrasonic():
    TRIG_PIN.write_digital(0)
    ECHO_PIN.set_pull(ECHO_PIN.NO_PULL)
```

```
# Measuring distance (cm)
def measure_distance():
    # A trigger signal of 10μs was sent
    TRIG_PIN.write_digital(1)
    utime.sleep_us(10)
    TRIG_PIN.write_digital(0)

    # Wait for the echo pin to get high
    timeout = 50000   # 50ms timeout
    start_time = utime.ticks_us()
    while ECHO_PIN.read_digital() == 0:
        if utime.ticks_diff(utime.ticks_us(), start_time) > timeout:
            return -1  # timeout

    # The echo onset time was recorded
    pulse_start = utime.ticks_us()

    # Wait for the echo pin to get low
    while ECHO_PIN.read_digital() == 1:
        if utime.ticks_diff(utime.ticks_us(), pulse_start) > timeout:
            return -1  # timeout

    # The echo end time was recorded
    pulse_end = utime.ticks_us()

    # Calculate the pulse duration
    pulse_duration = utime.ticks_diff(pulse_end, pulse_start)

    # Calculating distance（The Speed of Sound
    # 340m/s = 0.034cm/μs，Round trip, so divided by 2）
    distance = (pulse_duration * 0.034) / 2

    return distance

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
```

```
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

setup_ultrasonic()
display.show(Image.HAPPY)
sleep(400)

# Code in a 'while True:' loop repeats forever
while True:
    # measuring distance
    dist = measure_distance()
    sleep(200) # Necessary delay

    if dist > 10 and dist < 15:
        # Stop
        setWheelSpeed(leftwheel, forward, 0)
        setWheelSpeed(rightwheel, forward, 0)
        if random.randint(0, 1) == 0:
            # Turn left
```

```
            setWheelSpeed(leftwheel, backward, 50)
            setWheelSpeed(rightwheel, forward, 50)
        else:
            # Turn right
            setWheelSpeed(leftwheel, forward, 50)
            setWheelSpeed(rightwheel, backward, 50)
        sleep(200)

    else:
        # Forward
        setWheelSpeed(leftwheel, forward, 100)
        setWheelSpeed(rightwheel, forward, 100)
```

Conclusion



The car moves forward at full speed. When it detects an obstacle within 15cm, it randomly rotates left or right by an angle and continues forward.

Thinking

Common Issues

Q: When using the Pybit, it was found that the car was running normally initially, but after connecting the ultrasonic sensor, the car malfunctioned and could not drive.

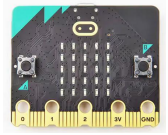
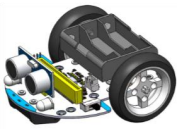
A: Please check if the ultrasonic sensor is plugged into the wrong interface; it should be plugged into the sonar interface, not the I2C interface.

6.6 Light seeking

Objective

The micro:bit mainboard's dot matrix can sense light intensity. Use this feature to make the Pybit automatically follow a light source in a dark environment.

Tools

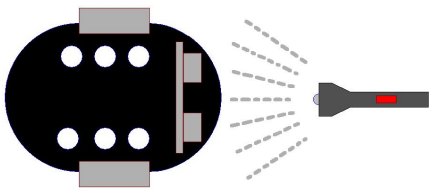
PC	micro:bit v2.x.x	Micro USB Cable	Pybit	Flashlight
				

Programming

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
```

```
        speed = 0
    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    if display.read_light_level() < 50:
        # Turn left at place
        setWheelSpeed(leftwheel, backward, 50)
        setWheelSpeed(rightwheel, forward, 50)
    else:
        # Forward
        setWheelSpeed(leftwheel, forward, 100)
        setWheelSpeed(rightwheel, forward, 100)
```

Conclusion



Shine a flashlight on the micro:bit mainboard's dot matrix. When the detected light level is less than 50, the car rotates in place. When the detected light level is greater than 50, the car moves forward at full speed.

Thinking

Why might this function not be achievable in a daytime environment?

Common Issues

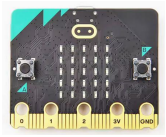
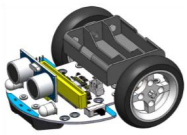
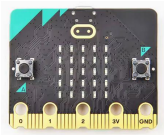
1. This lesson needs to be conducted in a dark environment; it cannot be achieved in a daytime environment.
2. If expanding the experiment yourself, you cannot use any function of the micro:bit dot matrix, otherwise the lesson effect cannot be achieved.

6.7 micro:bit Wireless Control

Objective

- Use another micro:bit mainboard as a remote control to wirelessly operate the Pybit.
- Both mainboards need to be programmed.。

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit	micro:bit Mainboard
				

Programming

Remote Contro Programming:

```
# Imports go at the top
from microbit import *
import radio

# Display image
display.show(Image.HAPPY)
sleep(400)

# Configure the radio
radio.config(group=1, power=3)
radio.on()
```

```
# Code in a 'while True:' loop repeats forever
while True:
    if button_a.was_pressed():
        radio.send('1')

    if button_b.was_pressed():
        radio.send('2')

    if button_a.is_pressed() and button_b.is_pressed():
        dradio.send('3')
```

Pybit Programming:

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
```

```
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Display image
display.show(Image.HAPPY)
sleep(400)

# Configure the radio
radio.config(group=1)
radio.on()

# Code in a 'while True:' loop repeats forever
while True:
    message = radio.receive()
    if message == '1':
        # Turn left at place
        setWheelSpeed(leftwheel, backward, 60)
        setWheelSpeed(rightwheel, forward, 60)
    elif message == '2':
        # Turn right at place
        setWheelSpeed(leftwheel, forward, 60)
        setWheelSpeed(rightwheel, backward, 60)
    elif message == '3':
        # Forward
        setWheelSpeed(leftwheel, forward, 100)
        setWheelSpeed(rightwheel, forward, 100)
        sleep(200)
```

Conclusion

- Press the A+B buttons on the remote micro:bit, the car moves straight.
- Press the A button on the remote micro:bit, the car turns left.
- Press the B button on the remote micro:bit, the car turns right.

Thinking

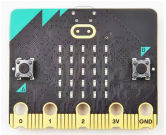
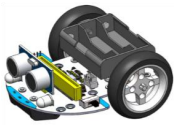
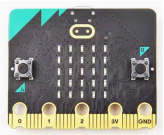
Common Issues

6.8 micro:bit Accelerometer Wireless Control

Objective

- Use another micro:bit's accelerometer to wirelessly control the Pybit, controlling the car's 360-degree direction and speed.
- Both mainboards need to be programmed.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit	micro:bit Mainboard
				

Programming

Remote Control Programming:

```
# Imports go at the top
from microbit import *
import radio

# Display image
display.show(Image.HAPPY)
sleep(400)

# Configure the radio
radio.config(group=1, power=3)
```

```
radio.on()

# Code in a 'while True:' loop repeats forever
while True:
    x_strength = accelerometer.get_x()/10
    radio.send('x' + str(x_strength))
    sleep(100)
    y_strength = accelerometer.get_y()/10
    radio.send('y' + str(y_strength))
    sleep(100)
```

Pybit Programming:

```
# Imports go at the top
from microbit import *
import radio

# Car I2C address
i2cAddr = 0x2a

# For wheels
leftwheel  = 0
rightwheel = 1
backward = 0
forward  = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05  # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
```

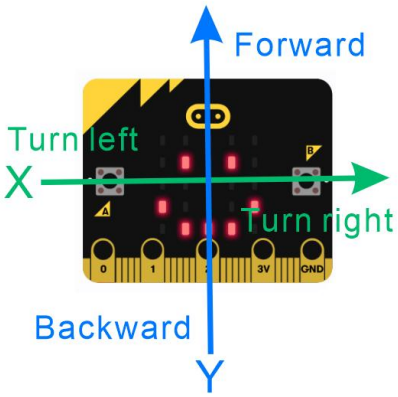
```
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06  # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Display image
display.show(Image.HAPPY)
sleep(400)

# Configure the radio
radio.config(group=1)
radio.on()

xData = 0
yData = 0

# Code in a 'while True:' loop repeats forever
while True:
    message = radio.receive()
    if message != None:
        # Gets the first character of a string.
        xy = message[0]
        # Gets the second to last character of a string.
        xyData = message[1:]
        # Converts a string to a number.
        xyData = int(xyData)

        if xy == 'x':
            xData = xyData
        if xy == 'y':
            yData = xyData

    if yData + xData >= 0:
        setWheelSpeed(leftwheel, forward, yData + xData)
    if yData + xData < 0:
        setWheelSpeed(leftwheel, backward, abs(yData + xData))
    if yData - xData >= 0:
```

```
        setWheelSpeed(rightwheel, forward, yData - xData)
    if yData + xData < 0:
        setWheelSpeed(rightwheel, backward, abs(yData - xData))
```

Conclusion

▶ Tilt the micro:bit remote control mainboard in a certain direction to control the Pybit's forward direction.

▶ The tilt angle of the micro:bit remote control mainboard controls the Pybit's speed.

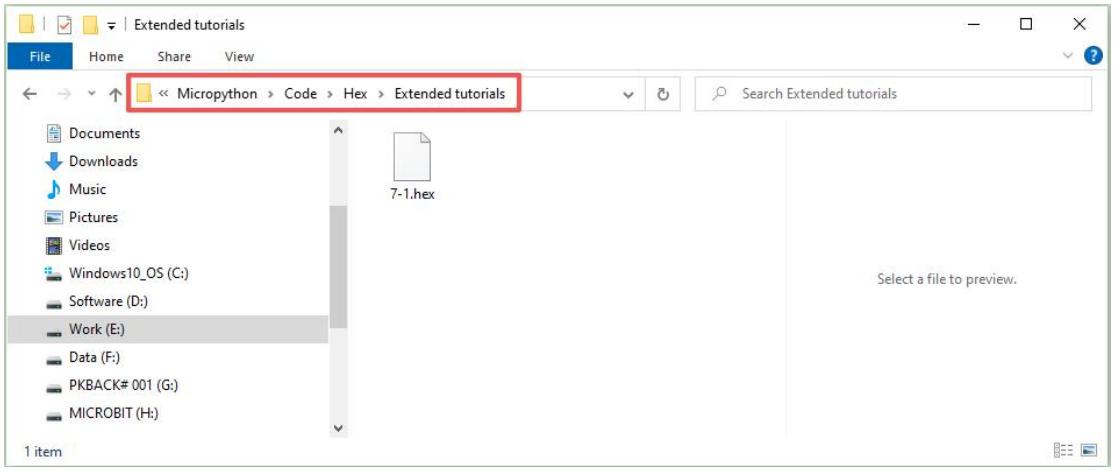


Thinking

Common Issues

7. Pybit Extended Tutorials

The example projects for the extended tutorials are all saved in the "Micropython -> Code -> Hex -> Extended tutorials" folder:

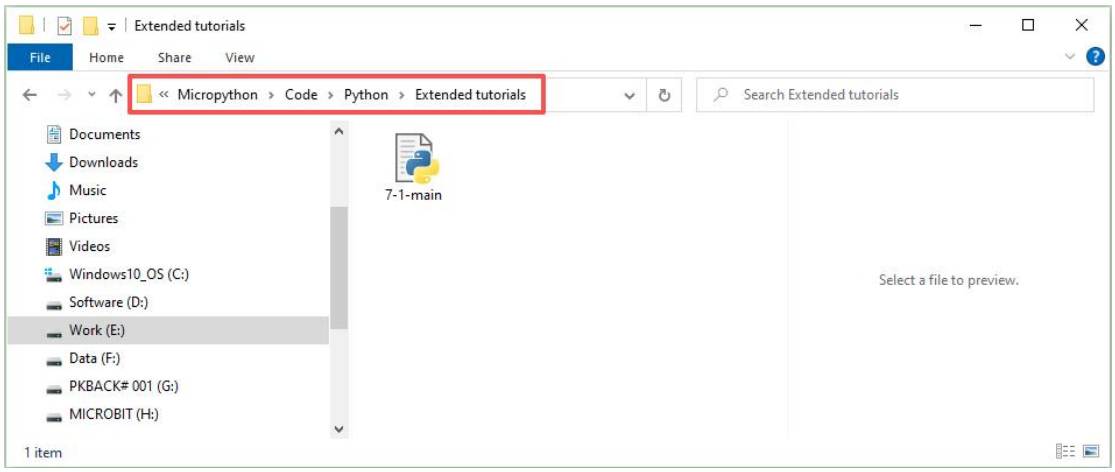


Special Reminder!!!

For the following tutorials, if you are importing our provided example projects, please refer to section 4.3. If you are creating a new project, please refer to section 4.1.

The Lego accessories used in the following lessons need to be purchased separately; they are not included in this kit!!!

The example Python files for the extended tutorials are saved in the "Micropython -> Code -> Python -> Extended tutorials" folder:

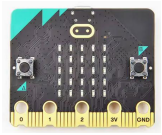
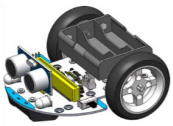


7.1 Driving Servos

Objective

Drive Lego servos or general-purpose servos.

Tools

PC	micro:bit v2.x.x	Micro USB Cable	Pybit	Lego Servo
				

Programming

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a
i2cBuf = bytearray([0x00, 0x00])

# Servo type
servo90  = 0
servo180 = 1
servo270 = 2
servo360 = 3
pinS1 = 0
pinS2 = 1
pinS3 = 2

# Set the Angle function of 90, 180 and 270 servo.
# index: 0 = S1 pin, 1 = S2 pin, 2 = S3 pin
# servoType: 0 = 90 servo, 1 = 180 servo, 2 = 270 servo
# angle: 90 servo -> 0-90, 180 servo -> 0-180, 270 servo -> 0-270
def setServo(index, servoType, angle):
    angleMap = 0
    if servoType == servo90:
        # Map 0-90 to 50-200
        angleMap = scale(angle, from_=(0, 90), to=(50, 200))
    if servoType == servo180:
        # Map 0-90 to 50-200
        angleMap = scale(angle, from_=(0, 180), to=(50, 200))
    if servoType == servo270:
        # Map 0-90 to 50-200
```

```
        angleMap = scale(angle, from_=(0, 270), to=(50, 200))

    if index == pinS1:
        i2cBuf[0] = 0x0d  # S1 pin
    if index == pinS2:
        i2cBuf[0] = 0x0e  # S2 pin
    if index == pinS3:
        i2cBuf[0] = 0x0f  # S3 pin

    i2cBuf[1] = angleMap
    i2c.write(i2cAddr, i2cBuf)

# Set the speed of 360 servo
# index: 0 = S1 pin, 1 = S2 pin, 2 = S3 pin
# speed: -100 to +100
def setServo360(index, speed):
    # Map -100 - 100 to 0 - 180
    angle = scale(speed, from_=(-100, 100), to=(0, 180))
    setServo(index, servo180, angle)

# Code in a 'while True:' loop repeats forever
while True:
    # The 180 degree servo of the S1 pin turns to the 0 degree position
    setServo(pinS1, servo180, 0)
    # The 360-degree servo of the S3 pin turns forward at the maximum speed
    setServo360(pinS3, 100)
    sleep(1000)
    # The 180 degree servo of the S1 pin turns to the 180 degree position
    setServo(pinS1, servo180, 180)
    # The 360-degree servo of the S3 pin turns backward at
    # the maximum speed
    setServo360(pinS3, -100)
    sleep(1000)
```

Conclusion

The servo cycles between 0 and 180 degrees.

Thinking

How to drive 90-degree and 270-degree servos?

Common Issues

8. QA

8.1 Cannot Upload Code to micro:bit

☛ Is a USB cable with data communication function being used?

☛ Is the USB cable in good condition?

8.2 micro:bit Drive Letter Display Incorrect

☛ The drive letter displays as: MAINTENANCE (Normal is MICROBIT). This happens because the micro:bit reset button was accidentally held down while powering on the micro:bit, causing it to enter firmware update mode. Re-update the firmware to return to normal. You can refer to the following link for operation:

https://microbit.org/get-started/user-guide/firmware/

8.3 Motors Still Rotate After Re-uploading Code

☛ This is because the function to stop the motors from section 5.6 was not called again:

```
setWheelSpeed(wheel, direction, 0)
```

8.4 Motors and Headlights Not Working

☛ Ensure the micro:bit is fully inserted into the Pybit's slot.

☛ Ensure the micro:bit edge connector is clean.

☛ Ensure the Pybit's slot is clean.

☛ Refer to the common issues in section 6.1 and reset the wheel compensation value to 0.

8.5 Other Faults

- Please check if the assembly is correct?
- Please check if the battery power is sufficient?
- Please check if the batteries used meet the specifications?

9. Contact US

We are committed to open-source hardware maker education, sincerely sharing our products with everyone, and hope to gain your recognition. We look forward to communicating more with you, identifying shortcomings in our products, and hope for frequent interaction and your participation in our team.

support@siyeenove.com

https://siyeenove.com